Registro de Tecnología y Propuesta Inicial

XS0131 Gestión de Bases de Datos y Análisis de Información

Escuela de Estadística — Grupo 8, duelo 7-8

Investigación Grupal 1

Desafío NoSQL

Caso 4: FraudeLink

Integrantes: Joselyn Godínez, Emilio Zamora, Carlos Quesada, Alejandro Castro, Giovanni Calvo Grillo

Caso asignado: Caso 4 - FraudeLink. Modelo NoSQL asignado: bases de datos de grafos. El equipo compite en el duelo correspondiente a este modelo contra el otro grupo asignado al mismo caso, cada uno con una tecnología distinta.

Tecnología seleccionada y versión prevista

Tecnología seleccionada: Memgraph, con la biblioteca de algoritmos MAGE, desplegado mediante contenedores Docker (imagen memgraph/memgraph-mage). Versión prevista: Memgraph 3.12.0. La interfaz de administración y consulta se realiza mediante Memgraph Lab (imagen memgraph/lab, versión 3.12.1), que se ejecuta como un segundo contenedor enlazado al motor principal.

Esta selección se registró en el foro de Mediación Virtual habilitado al inicio del trabajo, reservando el uso de Memgraph dentro del duelo correspondiente al Caso 4.

Justificación de correspondencia con el modelo NoSQL asignado

Memgraph es una base de datos de grafos de verdad, no una base de otro tipo que ofrece grafos como un extra. Toda la solución trabaja con nodos y relaciones, y todas las consultas se escriben en Cypher, el lenguaje de consultas de grafos. El caso FraudeLink necesita encontrar cuentas conectadas entre sí, dispositivos e IP compartidos, transferencias que forman ciclos y caminos entre cuentas sospechosas. Todo eso se expresa de forma directa como patrones en un grafo, por eso este modelo es el adecuado para el caso y no uno relacional o de documentos.

Descripción de la arquitectura propuesta y principales estructuras de datos

La solución tiene tres partes. Primero, un script en Python que genera los datos de prueba con la biblioteca Faker y una semilla fija (así siempre salen los mismos datos) y los carga en la base con el protocolo Bolt, usando el driver neo4j, que es compatible con Memgraph. Segundo, Memgraph

con MAGE, en una sola instancia dentro de un contenedor Docker, en modo IN_MEMORY_TRANSACTIONAL (los datos se guardan en memoria). Tercero, Memgraph Lab, que sirve para hacer consultas, ver el grafo y medir cuánto tarda cada consulta.

El esquema de datos tiene cinco tipos de nodo (Cliente, Cuenta, Dispositivo, IP y Comercio) y cinco relaciones con dirección:

- (Cliente)-[:POSEE]->(Cuenta)
- (Cuenta)-[:USA_DISPOSITIVO]->(Dispositivo)
- (Dispositivo)-[:CONECTA_DESDE]->(IP)
- (Cuenta)-[:TRANSFIERE_A {monto, fecha}]->(Cuenta)
- (Cuenta)-[:PAGA_EN {monto, fecha}]->(Comercio)

Cada nodo guarda solo lo necesario para identificarlo (un identificador y, en Cliente y Comercio, un nombre). El monto y la fecha de cada operación se guardan en las relaciones TRANSFIERE_A y PAGA_EN, y no en un nodo aparte de tipo Transferencia, porque ese nodo extra duplicaría el número de saltos en los recorridos de ciclos y caminos mínimos.

Plan de pruebas y riesgos técnicos identificados

Plan de pruebas

Para saber si las consultas funcionan no basta con ver que los resultados parezcan razonables. Se compararán contra algo que ya conocemos: el generador de datos incluirá patrones de fraude a propósito, con identificadores fijos y cantidades conocidas (un dispositivo y una IP compartidos por un grupo fijo de cuentas, y un número fijo de ciclos de transferencias de 3 a 5 saltos). Así, cada consulta de detección se evalúa con una pregunta simple: ¿encuentra exactamente el patrón que sembramos? Además, se cruzarán resultados entre requisitos distintos y se medirá el tiempo de cada consulta principal en la misma instancia, documentando el procedimiento para que se pueda repetir.

Riesgos técnicos identificados

- **Límite de memoria:** Memgraph correrá en Docker sobre macOS, y Docker Desktop le da solo una parte de la memoria RAM del equipo. Algunas consultas (las de caminos de largo variable sin límite, o las que guardan todos los resultados antes de ordenarlos) podrían gastar más memoria de la disponible con decenas de miles de relaciones.
- **Consultas lentas:** Las consultas que no pueden parar al encontrar unos pocos resultados, como las que ordenan antes de aplicar un LIMIT, tienen que revisar todos los ciclos o relaciones del grafo. Por eso pueden tardar mucho más que las consultas que buscan un dato puntual.
- **Confundir fraude con casualidad:** En un grafo de este tamaño, algunos patrones (ciclos o cuentas muy conectadas) pueden aparecer por azar con una frecuencia parecida a la de los fraudes sembrados. Por eso cada hallazgo se comparará con lo que se esperaría por azar antes de considerarlo evidencia de fraude.

Repositorio: https://github.com/Jossygg11/Investigacion_grupal_1